KSP 38-2-4 Zákeřná parkovací místa

Řešení:

Existuje Sářino parkovací místo vždy?

Definujme naší parkovací řadu pomocí pole P, kde pro každé i-té parkovací místo platí $P(i) \in N_{(0)}$. Aby Sářino parkovací místo v řadě existovalo, musí platit $P(i-1) \leq P(i) \geq P(i+1)$, kde $P(i)$ je ono hledané místo.

Z toho vyplývá, že správným řešením, je každý vrchol, rovina nebo polo-vrchol kde buď $P(i-1)$ nebo $P(i+1)$ jsou číslováním rovny $P(i)$, opačný konec pak bude nižší než obě čísla. Představit si to můžeme jako hory kde pro každé parkovací místo bude jeho číslování značit výšku.

V situacích, kdy sářino místo v daném $P(i)$ neexistuje, musí být porušena podmínka v zadání a tedy jedno nebo obě ze sousedních míst musí být větší než $P(i)$.

To ale znamená, že na některou ze stran parkoviště číslování musí stoupat, nebo klesat. A právě v bodě, kde se bude číslování lámat, tedy narazíme na vrchol, jsme našli Sářino místo. Všimněme si, že to bude platit i v situacích kdy číslování bude stoupat až do konce. Jelikož okraje mají pomyslně hodnoty 0, **vždy nejvyšší číslo parkoviště je to správné místo.** Pokud nejvyšších čísel je více, podmínka v zadání platí taktéž, takže to je také správně.

Proto Sářino místo existuje v řadě vždy.

Algoritmus pro nalezení parkovacího místa:

Cílem je najít takový pivot, pro který bude platit podmínka výše: $P(i-1) \leq P(i) \geq P(i+1)$

Pozor, tohle už trvá lineární čas

Na začátku si vytvoříme pole ***K_VAL*** s počtem prvků ***N***, tedy tolik kolik má Sářino parkoviště ***P*** parkovacích míst a jeho hodnoty nastavíme na -1. Do tohoto pole si na pozici ***i*** bude Sára ukládat hodnotu, kterou získala podhlédnutím se pod ***i***-té auto na parkovišti ***P.*** Takto učiní jen v případě, že si na této pozici ještě žádnou hodnotu nezapsala (aby se nedívala pod stejné parkovací místo 2x): ***K_VAL[i] = P[i]***

Budeme si také udržovat interval ve kterém víme, že se Sářino místo nachází pomocí hranic ***Left*** a ***Right.*** Také si budeme ukládat ***Pivot*** jako střed tohoto intervalu.

Na začátku algoritmu ošetříme situace pokud má parkoviště počet parkovacích míst ***N*** menších než 4. A to tak, že se podíváme pod všechna místa a porovnáním zjistíme u kterého z nich splňujeme podmínku pro Sářino místo.

7282412229016 Radek Zach 38-2-4 1/2

Pokud N >= 4, tak v první fázi podhledů pod auta se podíváme na první dvě a poslední dvě místa (***P[0],P[1], P[N-2], P[N-1]***). Jako u všech parkovacích míst i tyto si uložíme na stejné pozice do ***K_VAL***. Pokud pro P[0] nebo P[N-1] neplatí podmínka Sářina místa, parkovací místa budou číslované od krajů někam do středu vzestupně. V opačném případě pokud jedna z podmínek platí, vrátíme ji jako Sářino parkovací místo.

Pokud jsme tedy nenašli parkovací místo na krajích, binárně budeme půlit interval ve kterém by se mohlo nacházet. V cyklu v každé iteraci načteme do **K_VAL** pokud už nebyly načteny hodnoty z parkoviště na pozicích: **P[Pivot-1], P[Pivot], P[Pivot+1]** a zkontrolujeme, zda neodpovídají hledanému místu: **K_VAL[Pivot] >= K_VAL[Pivot-1] && K_VAL[Pivot] >= K_VAL[Pivot+1]**. Pokud odpovídají, cyklus přerušíme, jinak pokračujeme.

Dále v podmínkách rozhodneme na kterou stranu intervalu v hledání jít.
Pokud je **K_VAL[Pivot+1] > K_VAL[Pivot]** → jdeme doprava → **Left = Pivot+1**, **Pivot = (Left+Right) /2**
Jinak → jdeme doleva → **Right = Pivot-1, Pivot = (Left+Right) /2**

Asymptotická složitost v počtu podhledů pod auta v řadě je ***O(log n).*** A to proto, jelikož pokaždé zmenšíme interval **Left**, **Right** přibližně na polovinu.

Všimni si, že na každou iteraci si potřebuješ pamatovat jenom krajní body současného úseku a pivot a jeho vedlejší prvky.
Takže nepotřebuješ celé pomocné pole na ukládání hodnot, stačí si pamatovat pouze konstantní počet proměnných.

Algoritmus je jinak správný, takže strhnu jenom 2 body, protože mi to přijde spíš jako chyba z nezkušenosti s rozborem sublineárních algoritmů.

Když chceme mít sublineární algoritmus, tak se typicky používá teoretický model "Orákulus", kde se můžeme dívat na libovolné místo na vstupu, bez toho abychom museli načítat celý vstup. Takhle se vyvarujeme tomu, že by načítání vstupu bylo součástí asymptotické složitosti

Tohle nemusíš znát ani psát ve svých řešení, ale je potřeba si uvědomit, že v momentě když si přečteš *celý* vstup, tak jsi už provedl lineární počet operací.

9b

7282412229023 Radek Zach 38-2-4 2/2